Cursul 3

Grafică pe calculator

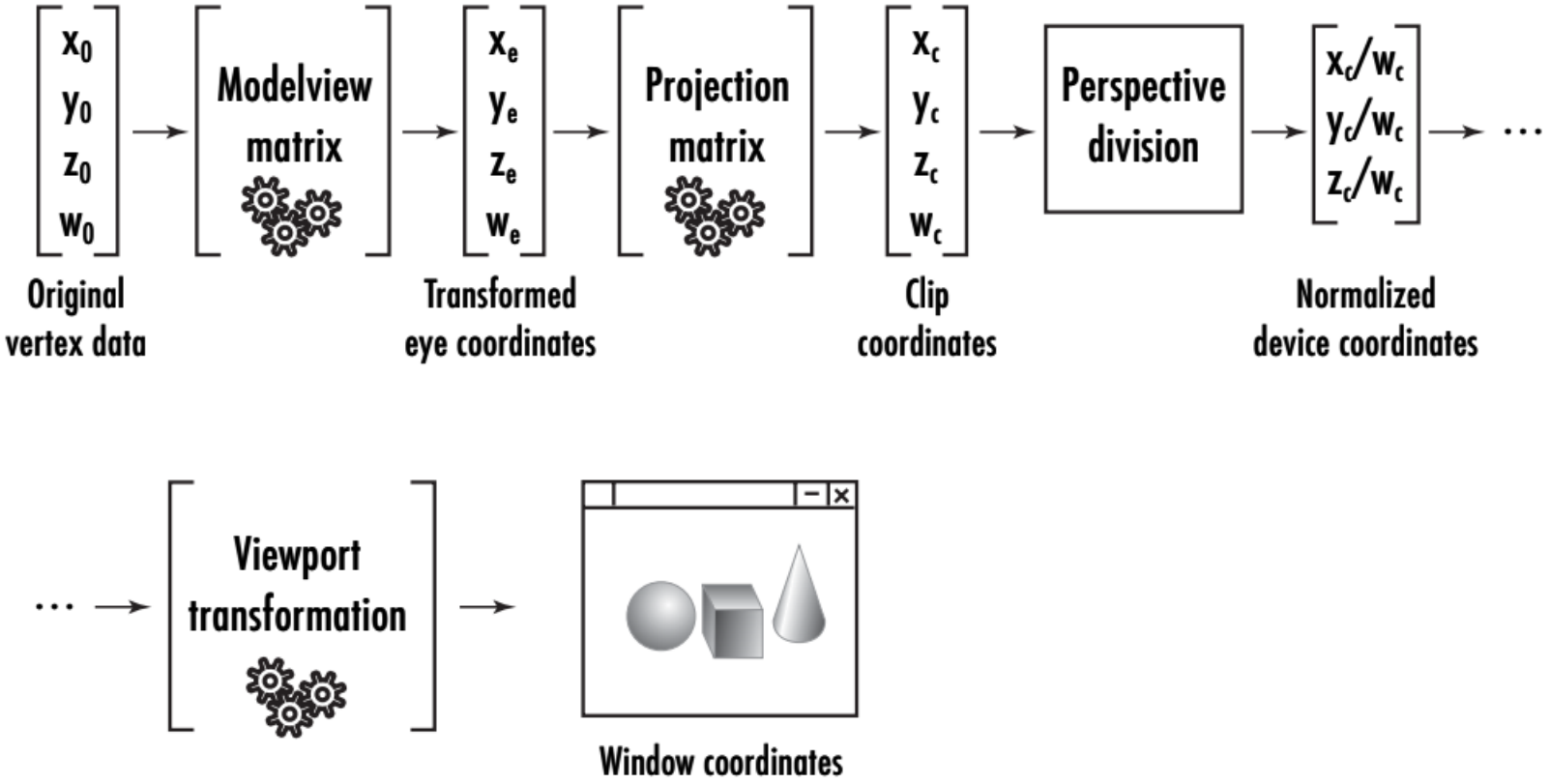
2025

Cuprins

1. Modelarea de obiecte 3D
   - Modelare poligonală
   - NURBS (Non-Uniform Rational B-Splines)
   - Alte tipuri de modelări
2. Exemplu de modelare
   - Exemple de formate de fișiere pentru obiecte 3D
3. Non-Uniform Rational B-Splines
4. Tehnici în compunerea unei scene 3D

Transformările vertex-urilor

OPENGL pipeline



Modelarea unui obiect 3D

Modelarea unui obiect în grafică pe computer implică crearea unei reprezentări digitale a unui obiect tridimensional (3D).

Metode de reprezentare:

- Modelare poligonală
- NURBS (Non-Uniform Rational B-Splines)
- Modelare implicită
- Modelare procedurală
- Modelare bazată pe voxeli

Modelare poligonală

Folosește vârfuri, muchii și fețe pentru a defini forma obiectului (de exemplu, triunghiuri și patrulatere).

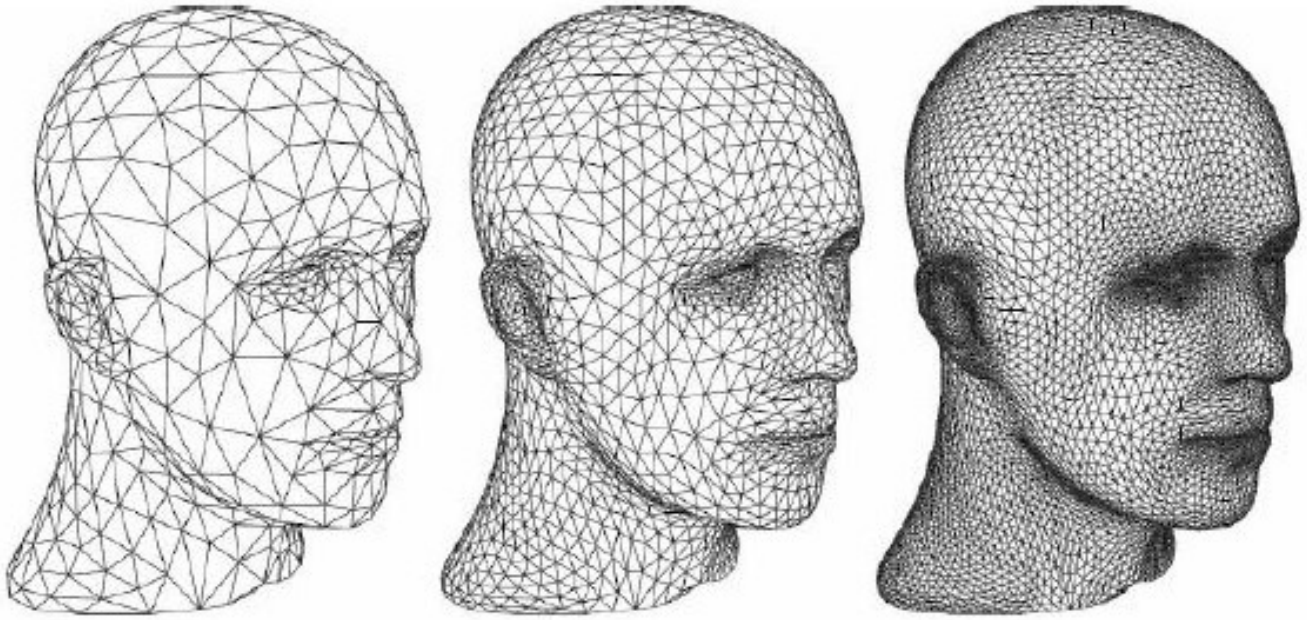


Figure: Image from: https://graficsweb.wordpress.com/2016/11/09/polygon-graphics/

Formatele comune includ .OBJ și .FBX.

NURBS (Non-Uniform Rational B-Splines)

Folosește curbe și suprafețe netede în loc de poligoane, utile pentru modelarea precisă.

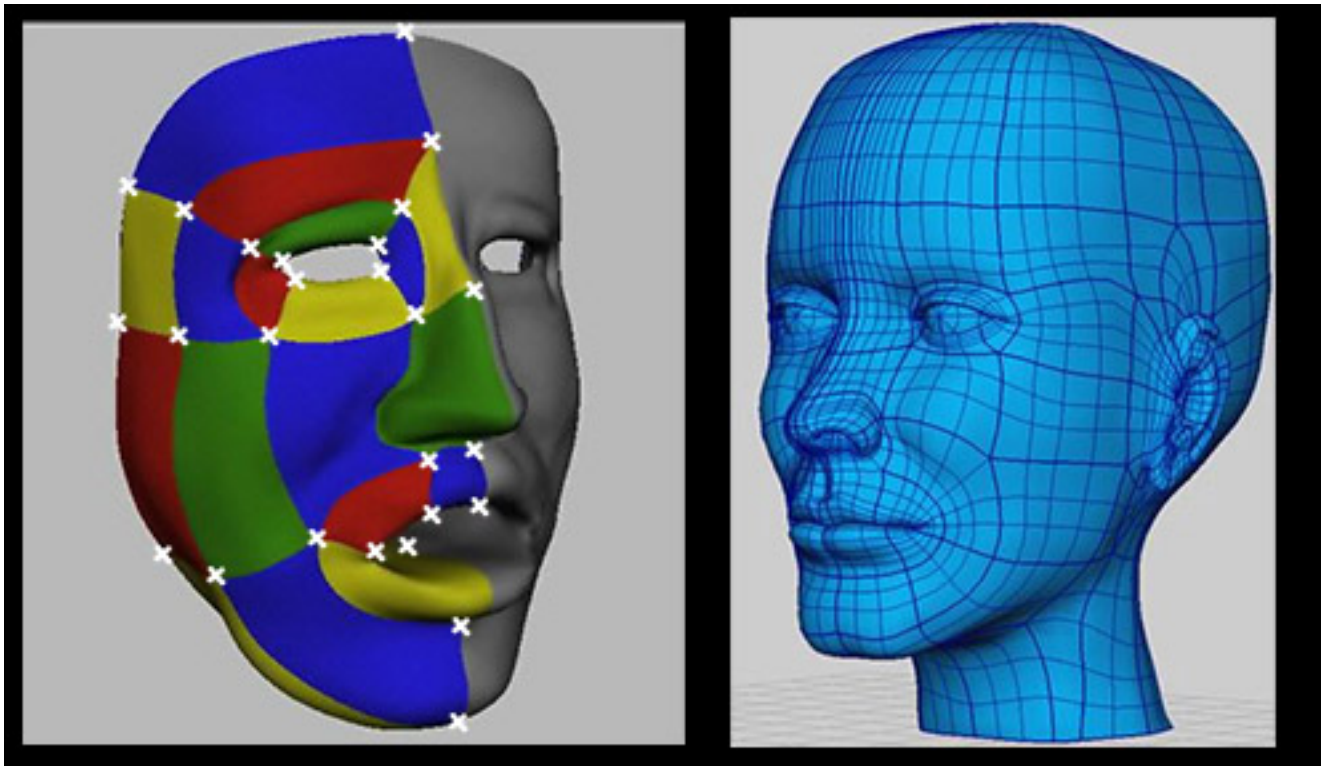


Figure: Image from: https://creativeclifton.wordpress.com/2012/11/10/nurbs/

Modelare implicită

Definește obiectele prin funcții matematice, cum ar fi metabile și izosuprafețele.

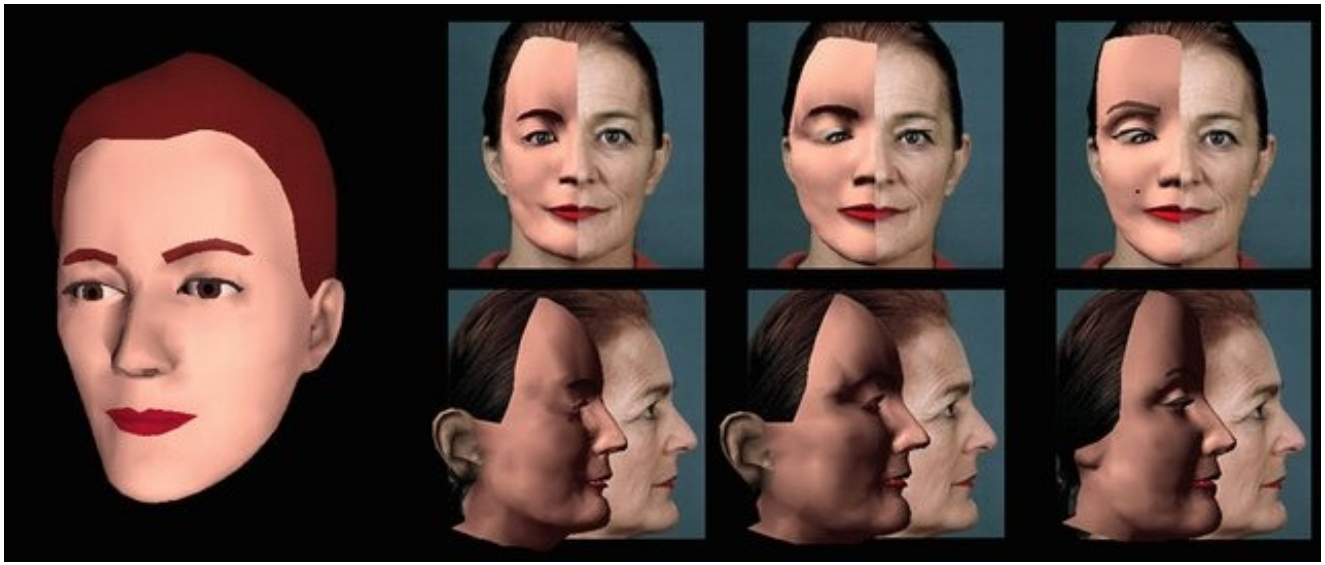


Figure: Image from: Kalra, Prem et.all, *Real-time animation of realistic virtual humans*, Journal of Computer Graphics and Applications, IEEE, vol. 18, October, 1998, p 42 - 56.

Modelare procedurală

Folosește algoritmi pentru a genera structuri complexe (de exemplu, fractali, generarea de teren pe bază de zgomot).

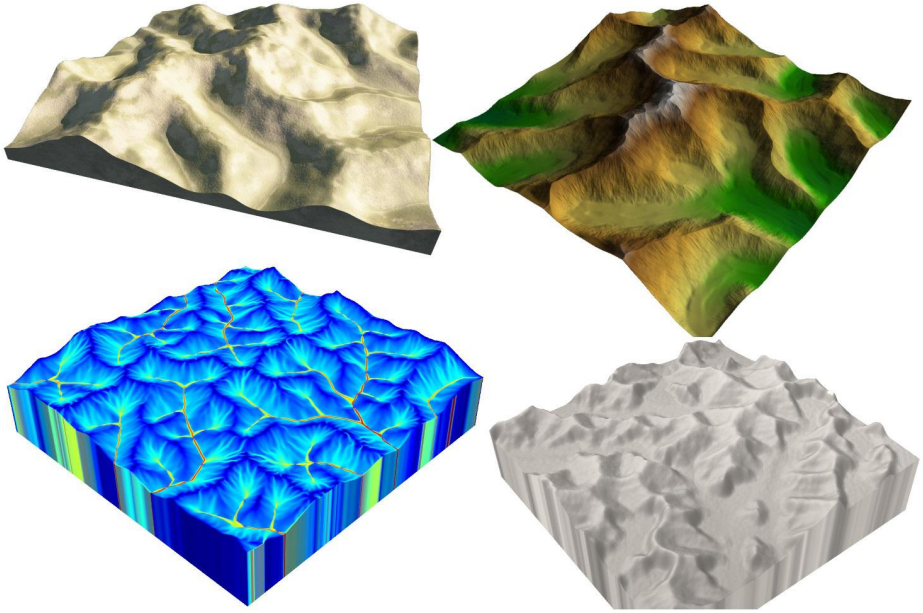


Figure: Image from: Gaillard, Mathieu, et al., *Dendry: A procedural model for dendritic patterns*, Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games. 2019.

Modelare bazată pe voxeli

Folosește o grilă de pixeli 3D (voxeli) pentru reprezentare, folosită în mod obișnuit în imagistica medicală și jocuri precum Minecraft.

Figure: Image from: https://www.freepik.com/

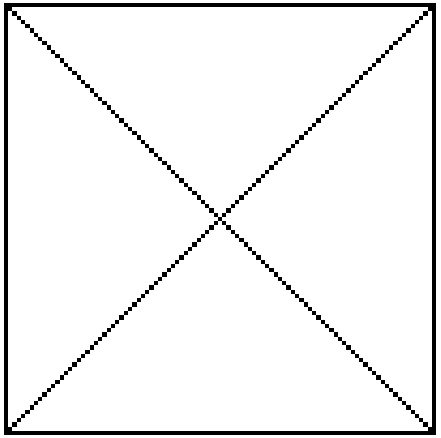
Modelarea unui cub

```
double vertex[] = {
 -0.5f,  0.5f,  0.5f,
  0.5f,  0.5f,  0.5f,
  0.5f,  0.5f, -0.5f,
 -0.5f,  0.5f, -0.5f,
 -0.5f, -0.5f,  0.5f,
  0.5f, -0.5f,  0.5f,
  0.5f, -0.5f, -0.5f,
 -0.5f, -0.5f, -0.5f
};
```

```
int triangles[] = {
    0, 1, 2,
    2, 3, 0,
    0, 1, 5,
    5, 4, 0,
    1, 2, 6,
    6, 5, 1,
    2, 3, 7,
    7, 6, 2,
    3, 0, 4,
    4, 7, 3,
    4, 5, 6,
    6, 7, 4};
```

Ce se vede

Fără nici o transformare!



Pentru a vedea cubul corect (în 3D) va trebui să mutăm camera și să creem o perspectivă.

O rotație de 45° după axa Oy

```
// Matrice de rotație în jurul axei Oy
Vec3 rotateY(Vec3 v, double angle) {
    double rad = angle * PI / 180.0;
    return {v.x * cos(rad) + v.z * sin(rad), v.y,
                    -v.x * sin(rad) + v.z * cos(rad)};
}
```

Amplasarea camerei

```
// Funcție pentru a aplica transformarea camerei
Vec3 transformCamera(Vec3 v, Vec3 camPos, Vec3 lookAt) {
    Vec3 forward = {lookAt.x - camPos.x, lookAt.y -
                        camPos.y, lookAt.z - camPos.z};
    double fLen = sqrt(forward.x * forward.x +
          forward.y * forward.y + forward.z * forward.z);
    forward = {forward.x / fLen, forward.y / fLen,
                                    forward.z / fLen};

    Vec3 up = {0, 1, 0};
    Vec3 right = {up.y * forward.z - up.z * forward.y,
                  up.z * forward.x - up.x * forward.z,
                  up.x * forward.y - up.y * forward.x};
    double rLen = sqrt(right.x * right.x + right.y *
                       right.y + right.z * right.z);
```

Amplasarea camerei

```
right = {right.x / rLen, right.y / rLen, right.z / rLen};

Vec3 newUp = {forward.y * right.z - forward.z * right.y,
              forward.z * right.x - forward.x * right.z,
              forward.x * right.y - forward.y * right.x};

Vec3 transformed = {
    (v.x - camPos.x) * right.x + (v.y - camPos.y) *
                    right.y + (v.z - camPos.z) * right.z,
    (v.x - camPos.x) * newUp.x + (v.y - camPos.y) *
                    newUp.y + (v.z - camPos.z) * newUp.z,
    (v.x - camPos.x) * forward.x + (v.y - camPos.y) *
                 forward.y + (v.z - camPos.z) * forward.z
};
return transformed;
```

Proiecția pe ecran

```
// Funcție de proiecție
Vec2 project(Vec3 v) {
    double d = 200.0; // Distanța de proiecție
    return {(int)(WIDTH / 2 + v.x * d / (d - v.z)),
                (int)(HEIGHT / 2 - v.y * d / (d - v.z))};
}
```

Softuri pentru modelarea de obiecte 3D

Există o multitudine de softuri care permit modelarea de obiecte 3D.

Exemple:

- Blender – program software liber de grafică 3D

  crearea modelelor 3D, mapare UV, texturare, rigging, simularea apei, animatie, randare, particule și alte simulări computerizate, editare non-lineară, compositing și crearea aplicațiilor interactive.

- Unity – un game engine

  crearea de jocuri și experiențe 2D și 3D (filme – cinematice, interactive, virtuale...)

alte exemple: Cinema 4D, 3ds Max Design, Modo, Maya, Animaker, Adobe Premiere Pro, Adobe After Effects.

Formate de fișiere pentru obiecte în 3D

modelul 3D creat este salvat în fișiere de diverse formate:

- Collada – .dae
- Alembic – .abc
- Universal Scene Description – .usd, .usdc, .usda
- Grease Pencil as SVC
- Stanford – .ply
- STL – .stl
- FBX – .fbx
- glTF 2.0 – .glb, .gltf
- WaveFront – .obj
- X3d Extensible 3D – .x3d

Exemplu: formatul glTF

formatul glTF™ (GL Transmission Format) este folosit pentru transmisia și încărcarea modelelor 3D în aplicații web

- reduce mărimea modelelor și timpii de procesare a lor (încărcare și desenare);
- este acceptat de diverse motoare 3D precum: Unity3D, Unreal Engine 4, and Godot;
- permite transmisia următoarelor date:
  - Meshes,
  - Materiale (Principled BSDF) și Shadeless (Unlit),
  - Texturi,
  - Camere,
  - Lumini punctuale (puncte, spot și direcționale),
  - Animații.

Exemplu: formatul .obj

Generalități

A fost dezvoltat inițial de Wavefront Technologies și a fost preluat și de alte companii.

E simplu și accesibil:

- Comentariile sunt marcate cu #
- Poate conține date legate de vertexuri, curbe, suprafețe, corpuri, moduri de grupare etc.

Elementele cele mai comune sunt cele geometrice:

vertex-uri, *coordonatele texturilor*, *normalele* și *fețele poligonale*.

Exemplu: formatul .obj

Exemplu de elemente din fișier

```
# List of geometric vertices, with (x, y, z [,w])
# coordinates, w is optional and defaults to 1.0.
v 0.123 0.234 0.345 1.0
v ...
...
# List of texture coordinates, in (u, [,v ,w])
#  coordinates, these will vary between 0 and 1.
#  v, w are optional and default to 0.
vt 0.500 1 [0]
vt ...
...
# List of vertex normals in (x,y,z) form; normals
# might not be unit vectors.
vn 0.707 0.000 0.707
vn ...
```

Exemplu: formatul .obj

Exemplu de elemente din fișier

```
# Parameter space vertices in ( u [,v] [,w] ) form;
# free form geometry statement
vp 0.310000 3.210000 2.100000
vp ...
...
# Polygonal face element (see below)
f 1 2 3
f 3/1 4/2 5/3
f 6/4/1 3/5/3 7/6/5
f 7//1 8//2 9//3
f ...
...
# Line element
l 5 8 1 2 4 9
```

Exemplu: formatul .obj

Descrierea formatului

Câteva elemente din formatul fișierelor *.obj*:

fiecare linie are la început o etichetă ce precizează tipul elementului de pe acea linie:

- **v** – vertex-uri, **vt** – coordonate texturi, **vn** – vectori normali
- **f** – fețe (pot avea mai multe formate):
  - listă de vertex-uri:
    $f\ v1\ v2\ v3;$
  - listă de vertex/texturi:
    $f\ v1/vt1\ v2/vt2\ v3/vt3;$
  - listă de vertex/texturi/normale:
    $f\ v1/vt1/vn1\ v2/vt2/vn2\ v3/vt3/vn3$
  - lista de vertex//normale:
    $f\ v1//vn1\ v2//vn2\ v3//vn3$

Non-Uniform Rational B-Splines

Față de B-Spline-urile clasice, NURBS oferă un control mai precis asupra formei prin utilizarea unor ponderi asociate fiecărui punct de control.

O curbă NURBS de grad p este definită prin formula:

$$C(t) = \frac{\sum_{i=0}^{n} N_{i,p}(t) w_i P_i}{\sum_{i=0}^{n} N_{i,p}(t) w_i}, \quad t \in [t_0, t_m] \tag{1}$$

unde:

- P_i sunt punctele de control.
- w_i sunt ponderile asociate fiecărui punct de control.
- $N_{i,p}(t)$ sunt funcțiile de bază B-Spline de grad p.

Funcțiile de bază B-Spline

Funcțiile de bază B-Spline de grad p sunt definite recursiv astfel:

$$N_{i,0}(t) = \begin{cases} 1, & \text{dacă } t_i \leq t < t_{i+1} \\ 0, & \text{altfel} \end{cases} \tag{2}$$

$$N_{i,p}(t) = \frac{t - t_i}{t_{i+p} - t_i} N_{i,p-1}(t) + \frac{t_{i+p+1} - t}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(t) \tag{3}$$

Ponderile w_i influențează contribuția fiecărui punct de control la forma curbei.

- Dacă $w_i > 1$, punctul de control trage curba mai aproape de el.
- Dacă $w_i = 1$, curba se comportă ca o B-Spline standard.
- Dacă $0 < w_i < 1$, influența punctului de control asupra curbei este mai slabă.
- Dacă $w_i = 0$, punctul de control este ignorat complet.

Exemplu Numeric

Să considerăm o curbă NURBS definită prin trei puncte:

$$P_0 = (0,0), \quad P_1 = (0.5,1), \quad P_2 = (1,0)$$

Dacă setăm ponderi:

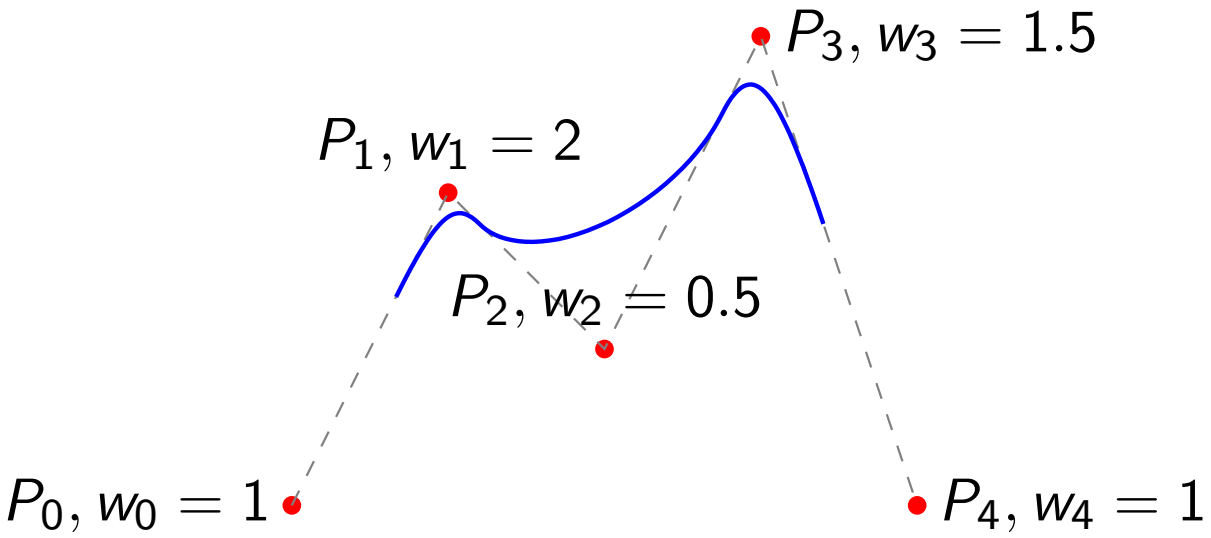
- $w_0 = 1$, $w_1 = 2$, $w_2 = 1 \rightarrow$ Curbă mai apropiată de P_1.
- $w_0 = 1$, $w_1 = 0.5$, $w_2 = 1 \rightarrow$ Curbă mai îndepărtată de P_1.

Interpolarea între două puncte cu ponderi diferite poate fi calculată astfel:

$$P(t) = \frac{(1-t)w_1 P_1 + t w_2 P_2}{(1-t)w_1 + t w_2} \tag{4}$$

Exemplu de NURBS

5 puncte în plan cu diverse ponderi



Tehnici în modelarea 3D

Modelare cu primitive: Folosește forme de bază (cuburi, sfere, cilindri) pentru a construi modele complexe.

Modelare prin Extrudare (Box Modeling): Începe cu o formă primitivă și o rafinează folosind extrudare, subdivizare și sculptare.

Sculptură Digitală (Sculpting): Utilizează unelte pentru a modela o rețea de mesh asemenea lutului (ex: în Blender, ZBrush).

Retopologie (Retopology): Transformă modelele sculptate cu rezoluție mare în versiuni optimizate cu un număr redus de poligoane.

Operații Booleene: Combină sau scade obiecte pentru a crea forme complexe.

Texturare și Materiale

Mapare UV: Proiectează o textură 2D pe un model 3D.

Shadere și Materiale: Definirea proprietăților suprafețelor, precum culoare, reflexie și rugozitate.

Randare și Iluminare

Modele locale pentru lumină: Simulează lumina folosind informații locale

Exemple: *modelul Phong*, *modelul Blinn-Phong*;

Modele globale pentru lumină: Simulează interacțiunile realiste ale luminii în medii 3D.

Exemplu: *Ray Tracing*

Software Utilizat în Grafica 3D

Blender: Software gratuit și open-source pentru modelare și animație.

Maya, 3ds Max (Autodesk): Software profesional utilizat în industria filmului și a jocurilor.

ZBrush: Specializat în sculptură digitală.

Cinema 4D, Houdini: Folosit pentru modelare procedurală și efecte vizuale.

Bibliografie

www.scratchapixel.com
www.opengl.org
learnopengl.com